

University of Idaho
College of Engineering

CS 466/566
PLC PROGRAMMING AND AUTOMATION
FALL 2024

LECTURE 4 – DIGITAL LOGIC II

INSTRUCTOR: DR JOHN SHOVIC
LECTURER: JACOB FRIEDBERG

DIGITAL LOGIC – 0'S AND 1'S

FOLLOWING ON FROM LECTURE 1

- Physical variables are continuous – voltage, frequency, mass, length, time - Analog
- Digital abstraction only looks at a discrete subset of values
  - 0 – Off, Low, False (0V)
  - 1 – On, High, True (5, 12, 24V)
  - Things are black and white with no grey area (mostly)
- Logic – Reasoning conducted based on principles of validity

ELECTRONIC COMBINATION LOCK

HOW IT WORKS

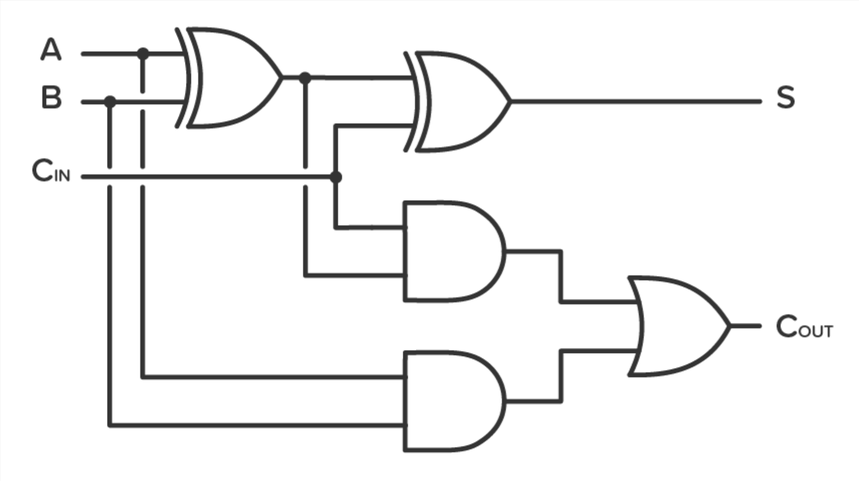
- Enter the pin – typically 4 digits, press pound (#) key
- If the pin is correct, unlock, otherwise do nothing and maybe provide some indication the pin was incorrect like a flashing LED
- Other possible functions:
  - Lockdown or alarm goes off after several failures
  - Program a new pin
  - Multiple pins possible
  - Physical override like a key
  - Logs of pins entered, failures, date and time
  - Smart/IoT locks – camera feed, app notifications

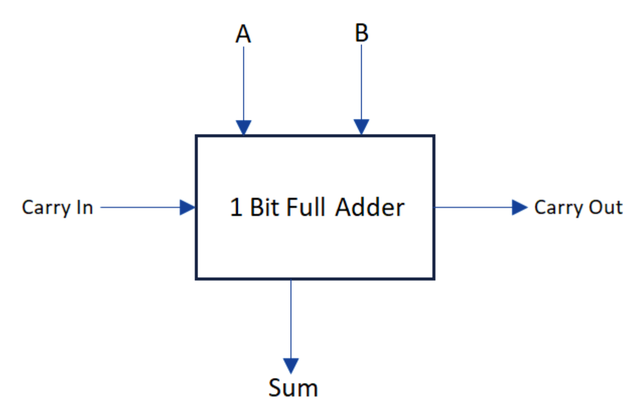


BUILDING A DIGITAL SYSTEM

- Requires analysis of the problem – inputs, expected outputs for a given input combination
- Designing a logic circuit that uses a discrete set of values
- Requires knowledge on digital number systems, binary operations, gates and building blocks made from these gates

Inputs			Outputs	
A	B	Carry In	Sum	Carry Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1





NUMBER SYSTEMS

- Basic unit is called a bit – **B**inary Digi**t**
- Only 2 possible values: 0 and 1
  - An n-bit binary number has 2^n possible values
- Decimal has 10 values: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Least Significant Bit (LSB) on right, Most Significant Bit (MSB) on left

COUNTING IN DECIMAL

1's column
10's column
100's column
1000's column

$$2\ 1\ 5\ 3_{10} = 2\times10^3 + 1\times10^2 + 5\times10^1 + 3\times10^0$$

COUNTING IN BINARY

1's column
2's column
4's column
8's column

$$1001_2 = 1\times2^3 + 0\times2^2 + 0\times2^1 + 1\times2^0$$

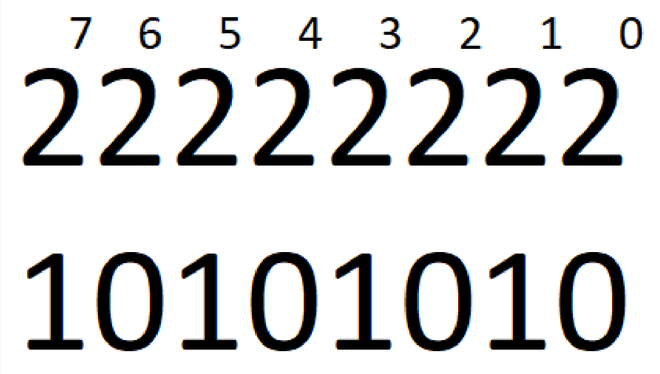
CONVERTING BINARY TO DECIMAL

METHOD

- Align the binary value with its column values
- Multiply the column value by its corresponding binary value
- Sum up the numbers
- Remember this sequence for 2^8 to 2^0: 256, 128, 64, 32, 16, 8, 4, 2, 1

Example

- 10101010 $=1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 128 + 32 + 8 + 2 = 170$



CONVERTING BINARY TO DECIMAL

EXAMPLES

- $110 = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 4 + 2 + 0 = 6$
- $10010 = 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 16 + 0 + 0 + 2 + 0 = 18$
- $10101010 = 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 128 + 32 + 8 + 2 = 170$
- $111001110 = 1 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 256 + 128 + 64 + 8 + 4 + 2 = 462$
- $1000110011 = 1 \times 2^9 + 0 \times 2^8 + 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 512 + 32 + 16 + 2 + 1 = 563$

CONVERTING DECIMAL TO BINARY

SUBTRACTION METHOD

- Given a decimal number, subtract the largest power of two(2^n) that fits(equal to or less than) in the number and record a 1. Repeat with the previous power of two(2^{n-1}) and the remainder from the subtraction. Repeat with smaller and smaller powers until you reach 2^0. If a power of two does not fit in the remainder, write a 0 and move on to the next power. Results read top down

DIVISION METHOD

- Divide the number repeatedly by 2 and record the remainder– alternatively even number = 0, odd number = 1
- Stop when you get to 1/2 or 0/2
- The result will read from bottom up

CONVERTING DECIMAL TO BINARY

EXAMPLES

- 150 – Subtraction
  - 150 - 128 = 22(1)
  - 22 - 64 (0)
  - 22 - 32 (0)
  - 22 - 16 = 6 (1)
  - 6 - 8 (0)
  - 6 - 4 = 2 (1)
  - 2 - 2 = 0 (1)
  - 0 - 1 (0) => 10010110 Read from top to bottom

CONVERTING DECIMAL TO BINARY

EXAMPLES

- 150 – Division
  - 150 / 2 = 75r0 (0)
  - 75 / 2 = 37r1 (1)
  - 37 / 2 = 18r1 (1)
  - 18 / 2 = 9r0 (0)
  - 9 / 2 = 4r1 (1)
  - 4 / 2 = 2r0 (0)
  - 2 / 2 = 1r0 (0)
  - 1 / 2 = 0r1 (1) => 10010110 Read from bottom to top

CONVERTING DECIMAL TO BINARY

EXAMPLES

- 3
- 10
- 150
- 255
- 256

CONVERTING DECIMAL TO BINARY

EXAMPLES

- 3
  - Subtraction 3-2 = 1 (1) => 1-1=0 (1) => 11
  - Division 3/2 = 1r1 (1) => ½ => 1r0 (1) => 11
- 10
  - Subtraction 10-8 = 2 (1), 2-4 => 0, 2-2 = 0 (1), 0-1 (0) => 1010
  - Division 10/2 = 5 (0), 5/2 = 2r1 (1), 2/2 = 1r0 (0), 1/2 = 0r1 (1) => 1010
- 150
  - Subtraction 150-128 = 22(1), 22-64 (0), 22-32(0), 22-16 = 6(1), 6-8 (0), 6-4 = 2(1), 2-2 = 0(1), 0-1 (0) => 10010110
  - Division 150/2 = 75 (0), 75/2 = 37r1 (1), 37/2 = 18r1 (1), 18/2 = 9(0), 9/2 = 4r1 (1), 4/2 = 2(0), 2/2 = 1(0), 1/2 =1r1 (1) => 10010110

CONVERTING DECIMAL TO BINARY

EXAMPLES

- 255
  - Subtraction 255-128 = 127 (1), 127-64 = 63(1), 63-32 = 31(1), 31-16 = 15(1), 15-8 = 7(1), 7-4 = 3(1), 3-2 = 1(1), 1-1=0 (1) => 11111111
  - Division 255/2 = 127r1 (1), 127/2 = 63r1 (1), 63/2 = 31r1 (1), 31/2 = 15r1 (1), 15/2 = 7r1 (1), 7/2 = 3r1 (1), 3/2 = 1r1 (1), 1/2 = 0r1 (1) => 11111111
- 256
  - Subtraction 256–256 = 0 (1), 0-128 (0), 0-64 (0), 0-32 (0), 0-16 (0), 0-8 (0), 0-4 (0), 0-2 (0), 0-1 (0) => 100000000
  - Division 256/2 = 128 (0), 128/2 = 64 (0), 64/2 = 32 (0), 32/2 = 16 (0), 16/2 = 8 (0), 8/2 = 4 (0), 4/2 = 2 (0), 2/2 = 1 (0), 1/2 (1) => 100000000

NEGATIVE NUMBERS - SIGNED

PREVIOUS EXAMPLES WERE UNSIGNED INTEGERS – ONLY POSITIVE

- Three ways of representing signed numbers – 1's complement, 2's complement and signed magnitude
- Only going to use 2's complement in this class
- Value of MSB of n length number is -2^{n-1} - a leading 1 indicates a negative number
- 1's complement
  - To get the negative of a number flip all bits – 0110 = 6 => 1001 = -6
- 2's complement
  - To get the negative number flip all bits then add 1 – 0110 = 6 -> 1001 +1 -> 1010 = -6
  - Need to go over arithmetic to understand better
- Signed Magnitude
  - Add a 1 in front of the number to get the negative – 0110 = 6 -> 1110 = -6

1’S COMPLEMENT VS 2’S COMPLEMENT

Binary	1’s Complement Decimal Value	2’s Complement Decimal Value	Signed Magnitude	Unsigned Decimal Value
0111	7	7	7	7
0110	6	6	6	6
0010	2	2	2	2
0001	1	1	1	1
0000	0	0	0	0
1000	-7	-8	-0	8
1001	-6	-7	-1	9
1110	-1	-2	-6	14
1111	-0	-1	-7	15

1’S COMPLEMENT VS 2’S COMPLEMENT

- 1’s complement had -0, signed magnitude did also
- Wasted a bit sequence on a number not needed
- Range of 1’s complement values: $[-2^{n-1}-1, 2^{n-1}-1]$
- 2’s complement uses all bit sequences
- Range of 2’s complement values: $[-2^{n-1}, 2^{n-1}-1]$
  - E.g. 8-bit 2’s complement numbers have the range [-128, 127]

BITS, BYTES, WORDS

- Each bit requires additional hardware to store
- Bits are grouped together to form words
  - 8 bit = byte – 256 values
  - 4 bit = nibble – 16 values
  - 16 bit = word - 2^{16} = 65,536
  - 32 bit = double word — 2^{32} = 4,294,967,296
  - 64 bit = quad word - 2^{64} = 18,446,744,073,709,551,616
- Naming of the words varies across computer architectures
  - Will cover naming again in OpenPLC and DoMore Designer later

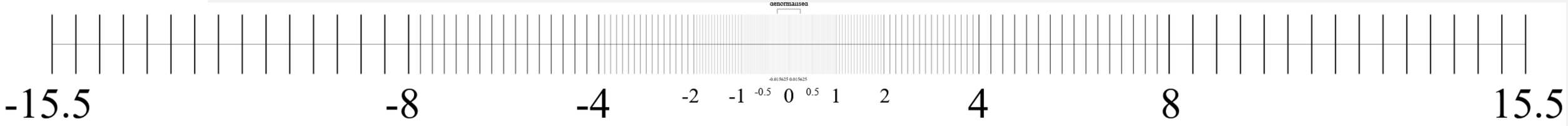
OTHER REPRESENTATIONS

- Hexadecimal – goes from 0-F – 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
  - Represents all sequences for 4 bits – 16 values – base 16
  - 2 Hexadecimal values is 1 byte e.g. FF = 11111111 = 255 or FF = $15 \times 16^1 + 15 \times 16^0 = 240 + 15 = 255$
  - Prefixed with 0x to indicate a hex value – 0xFF

Decimal	Hex	Binary	Decimal	Hex	Binary
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	10	A	1010
3	3	0011	11	B	1011
4	4	0100	12	C	1100
5	5	0101	13	D	1101
6	6	0110	14	E	1110
7	7	0111	15	F	1111

OTHER REPRESENTATIONS

- Fixed point – a way to represent real numbers and not just integers
  - Value of each bit after the decimal point represents ½ the previous value
  - $1.11 = 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 1 + 0.5 + 0.25 = 1.75$
- Floating point – greatly increased range of values at cost of precision
  - 32-bit Single Precision FP consists of a sign (1 bit), exponent (8 bits) and mantissa/significand (23 bits)
  - Can represent between $\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ to about 7 decimal digits
  - Not every value can be represented – 8-bit FP number line [3]



  - 64-bit (double precision), 128-bit (quadruple), 256-bit (octuple) versions exist

BINARY ARITHMETIC

THINK BACK TO ELEMENTARY/PRIMARY SCHOOL ARITHMETIC

$$\begin{array}{r} 168 \\ 502 \\ +\ 521 \\ \hline \end{array} \qquad \begin{array}{r} 888 \\ 415 \\ +\ 903 \\ \hline \end{array}$$

- 1’s column, 10’s column, 100’s column
- Add all the ones, if sum is between 10 and 19, carry a 1, if between 20 and 29, carry a 2 etc
- Repeat for the 10’s column, don’t forget to add the carry
- For subtraction, borrow from the next significant bit
- Same principle for binary

BINARY ARITHMETIC

ADDITION

- 10110110 + 00011000
- 01111111 + 10000001
- 10101010 + 01010101
- 35 + 70
- -50 + 63

BINARY ARITHMETIC

ADDITION

- 10110110 + 00011000

$$\begin{array}{r} 10110110 \\ +\ 00011000 \\ \hline 11001110 \end{array}$$

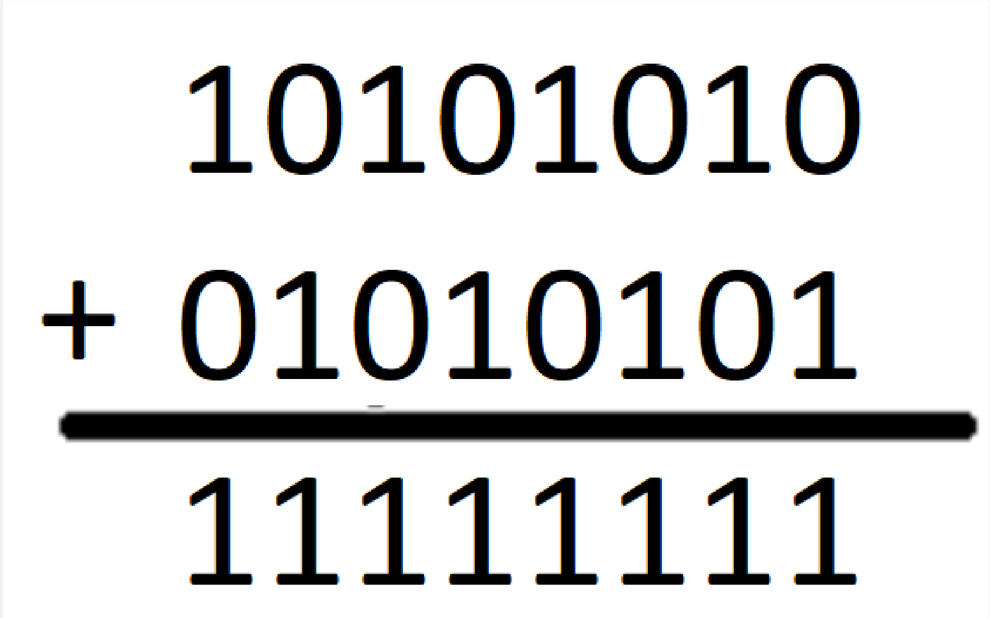
- 01111111 + 10000001

$$\begin{array}{r} 01111111 \\ +\ 10000001 \\ \hline 100000000 \end{array}$$

BINARY ARITHMETIC

ADDITION

- 10101010 + 01010101



BINARY ARITHMETIC

ADDITION

- 35 + 70
  - Convert to binary
  - 35 - 32 = 3 (1), 3-16 (0), 3-8 (0), 3-4 (0), 3-2 = 1 (1), 1-1 = 0 (1) => 100011
  - 70 – 64 = 6 (1), 6-32 (0), 6-16 (0), 6-8 (0), 6-4 = 2 (1), 2-2 = 0 (1), 0-1 (0) => 1000110
- Convert back to decimal to verify
- We know decimal answer is 105
- $1101001 = 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

= 64 + 32 + 8 + 1 = 105

$$\begin{array}{r} 100011 \\ +\ 1000110 \\ \hline 1101001 \end{array}$$

BINARY ARITHMETIC

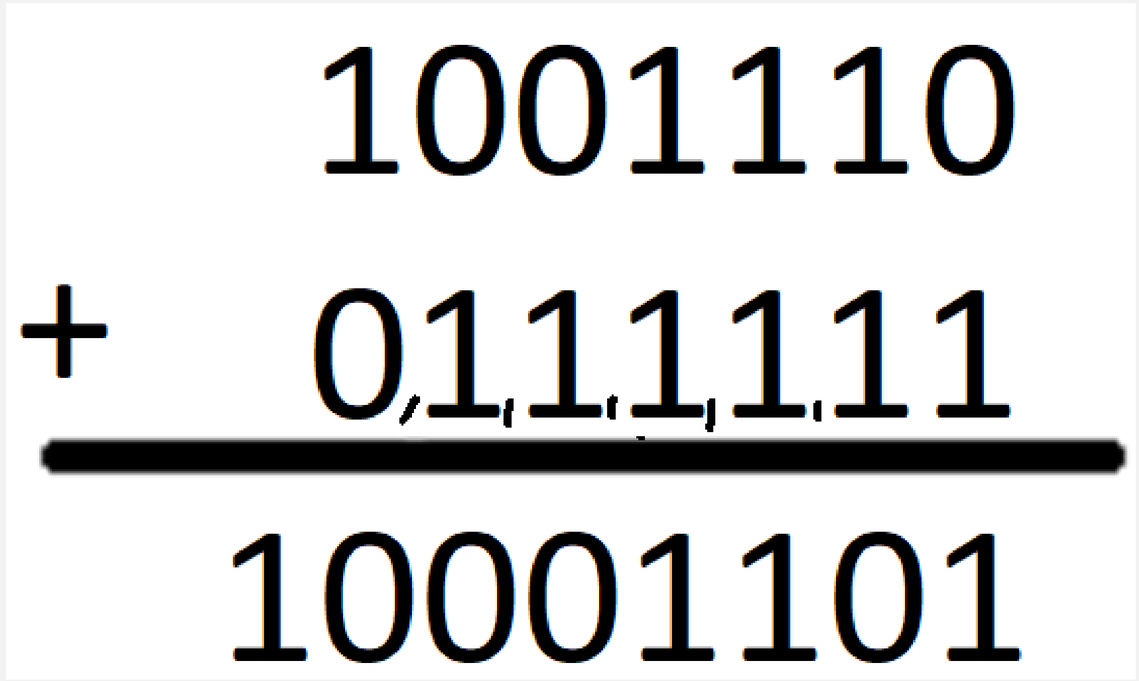
ADDITION

- -50 + 63
- Haven’t done binary conversion of a negative number yet
- Convert to positive binary value, then take 2’s complement – flip bits, then add 1
- 50 – 32 = 18 (1), 18 – 16 = 2 (1), 2-8 (0), 2-4 (0), 2-2 = 0 (1), 0-1 (0) => 110010
- This is a signed number, so add a leading 0 for the sign => 0110010
- Convert to -50, flip bits => 1001101, add 1 => **1001110**
- 63 to binary: every number that is 1 below 2^n is all 1’s => 111111
- Add a leading 0 => **0111111**

BINARY ARITHMETIC

ADDITION

- -50 + 63
- The 1 in the MSB column is discarded
- Convert back to decimal to check
- 1101 => 13



BINARY ARITHMETIC

SUBTRACTION

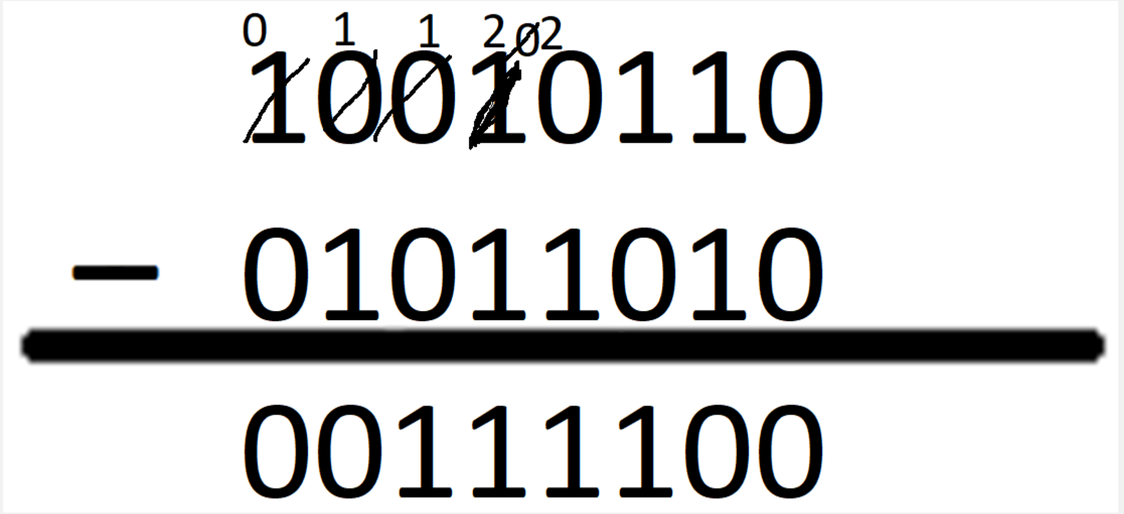
- For unsigned numbers, you must borrow from the next significant bit if you have a 0-1
- You cannot subtract a larger number from a smaller one
- Use 8-bit unsigned integer for this calculation
- 150 - 90

BINARY ARITHMETIC

UNSIGNED SUBTRACTION

- 150 - 90
- 150 => 10010110
- 90 => 01011010

- 00111100 = $2^5 + 2^4 + 2^3 + 2^2$
- $= 32 + 16 + 8 + 4 = 60$



BINARY ARITHMETIC

SIGNED SUBTRACTION

- For signed numbers, take the 2’s complement of the number being subtracted, then add
- Use 8-bit signed numbers for these calculations
- 35 – 70 will become 35 + -70
- 50 – 63 becomes 50 + -63

BINARY ARITHMETIC

SIGNED SUBTRACTION

- 35 – 32 = 3 (1), 3 – 16 (0), 3 – 8 (0), 3 – 4 (0), 3 – 2 = 1 (1), 1 – 1 = 0 (1) => 00100011
- 70 – 64 = 6 (1), 6 – 32 (0), 6 – 16 (0), 6 – 8 (0), 6 – 4 = 2 (1), 2 – 2 = 0 (1), 0 – 1 (0) => 01000110
- Take 2's complement of 70 => 10111001 + 1 => 10111010
- Verify result
- $-1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3$
- $+1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$
- = -128+64+16+8+4+1
- = -35

$$\begin{array}{r} 00100011 \\ +\ 10111010 \\ \hline 11011101 \end{array}$$

BINARY ARITHMETIC

SIGNED SUBTRACTION

- 50 – 63
- 50 => 00110010
- 63 => 00111111 => -63 = 11000001
- $-1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
- $= -128 + 64 + 32 + 16 + 2 + 1$
- = -13

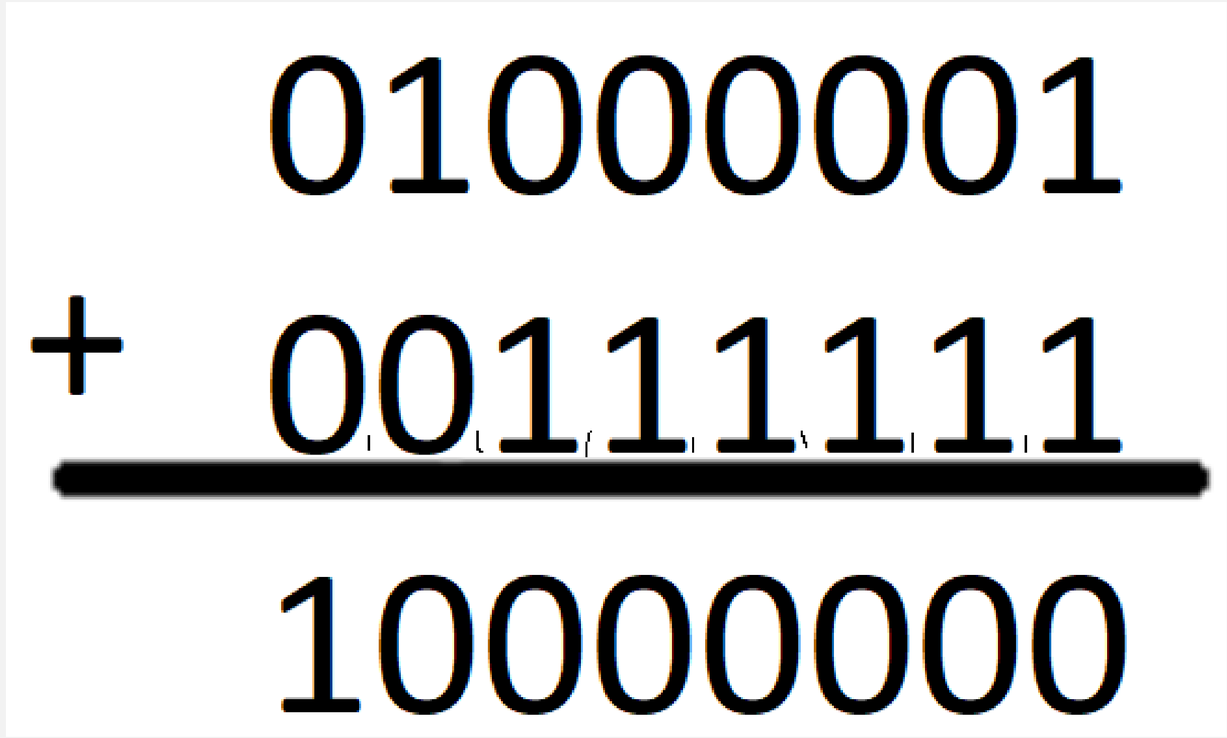
$$\begin{array}{r} 00110010 \\ + \quad 11000001 \\ \hline 11110011 \end{array}$$

OVERFLOW AND UNDERFLOW

- What happens with the following 8-bit signed addition?
- 65+63
- 65 = 01000001
- 63 = 00111111

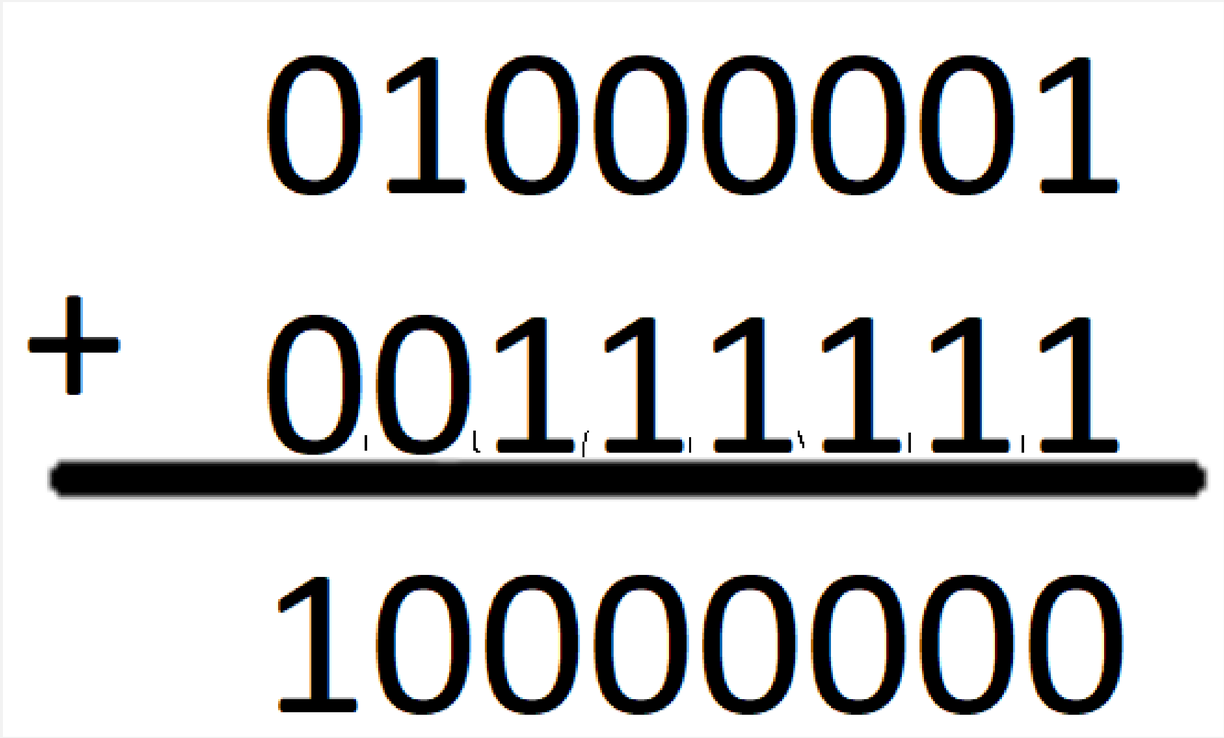
OVERFLOW AND UNDERFLOW

- 01000001 + 00111111 = 10000000
- = $-2^7 = -128$
- Result is -128
- What happened?



OVERFLOW AND UNDERFLOW

- 01000001 + 00111111
- Result is -128
- What happened?
- The maximum positive value was reached and rolled over to the maximum negative value
- How can you tell if overflow happened?



OVERFLOW AND UNDERFLOW

RULES

- Adding a negative number to a positive number never results in overflow or underflow
  - The carry out on the MSB is not an overflow/underflow error
- Adding 2 negative or 2 positive numbers can result in overflow or underflow
- Look at the sign bit of the two numbers being summed and their result.
- If the numbers are the same sign, but the result has a different sign = **OVERFLOW or UNDERFLOW**
- Adding 2 positive numbers and get a negative result – overflow
- Adding 2 negative numbers and get a positive result - underflow

OVERFLOW AND UNDERFLOW

CONSEQUENCES

- You may encounter an overflow or underflow when programming the PLCs and using counters
- Unexpected things may happen depending on how you programmed the logic
- Avoid by using an appropriate word size (8-bit, 16-bit) and type of number (signed, unsigned)

OTHER ARITHMETIC OPERATIONS

- Multiplication
- Division
- Shifting
  - Move all bits left 1 place – multiply by 2
  - Move all bits right 1 place – divide by 2

HOMEWORK 2

- Due Friday 9/8/22
- Practice binary/hex to decimal conversions and back
- Binary addition and subtraction
- Overflow and underflow detection
- Show all work
- Scan/take photo and submit to canvas

DIGITAL LOGIC 2

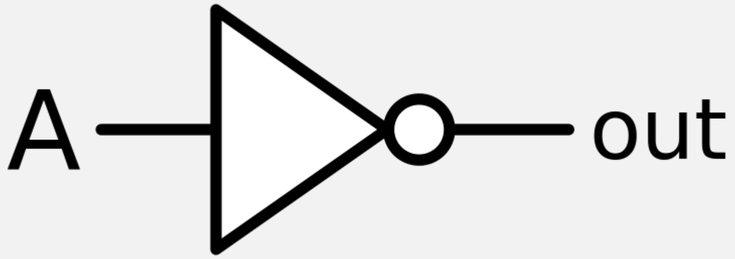
- Boolean Logic
  - Logic Gates
  - Combinational Logic
  - Simplification
  - Sequential Logic
- Digital constructs
  - Adders
  - Comparators
  - Counters
- State Machines
  - Digital Design

LOGIC OPERATIONS

LOGICAL OPERATIONS CAN BE DONE GIVEN 1 OR MORE INPUTS

- Truth tables show all possible combinations and outcomes
- Logic gates are a graphical representation
- Inputs usually called A, B, C, D
- Outputs usually called X, Y, Z, Q, Out – no hard rule on these
- NOT – Output is the opposite of the input - 1-> 0, 0->1
- Also called an Inverter
- Can be notated as ! A or $\bar{A}$ - will use the latter in examples

A	Out
0	1
1	0

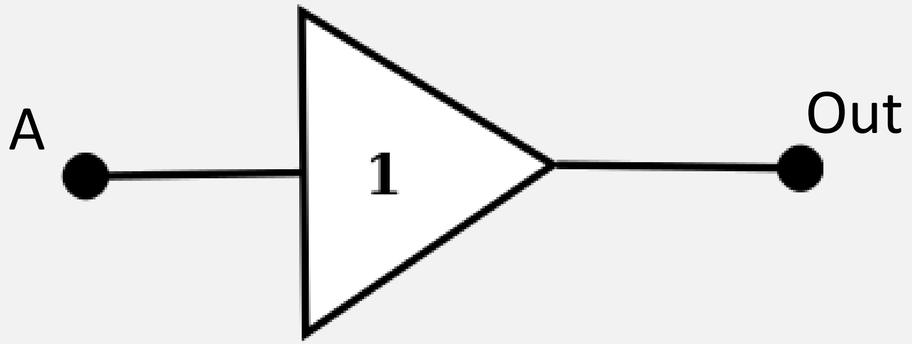


LOGIC OPERATIONS

BUFFER

- What goes in, also goes out

A	Out
0	0
1	1

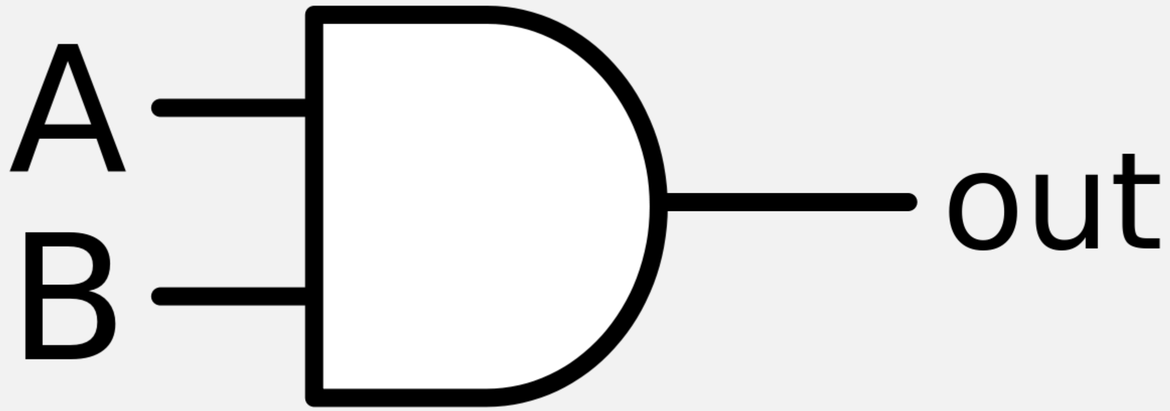


LOGIC OPERATIONS

- AND – Output is true if and only if all inputs are true

- Notated as AB or $A \cdot B$

A	B	Out
0	0	0
0	1	0
1	0	0
1	1	1

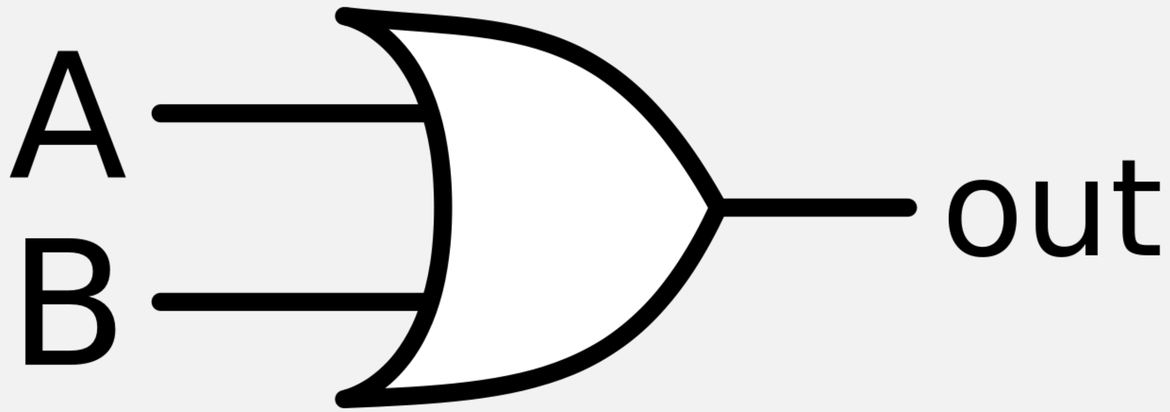


LOGIC OPERATIONS

- OR – Output is true if any input is true

- Notated as $A + B$

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1



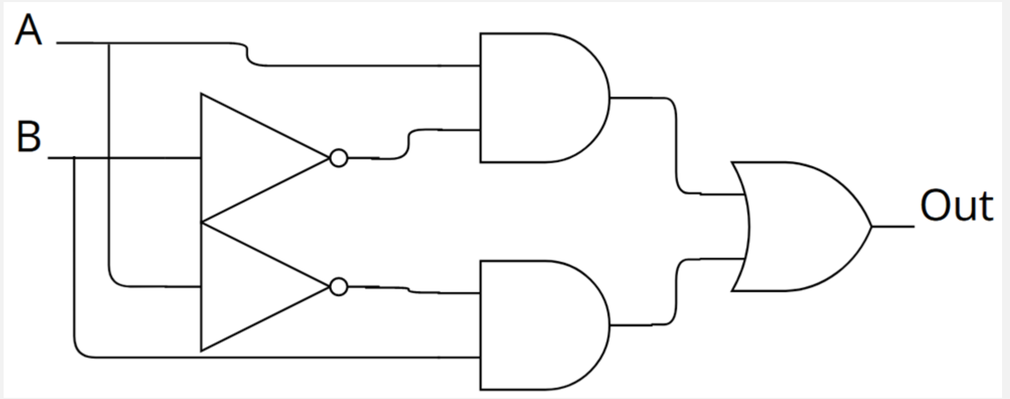
LOGIC OPERATIONS

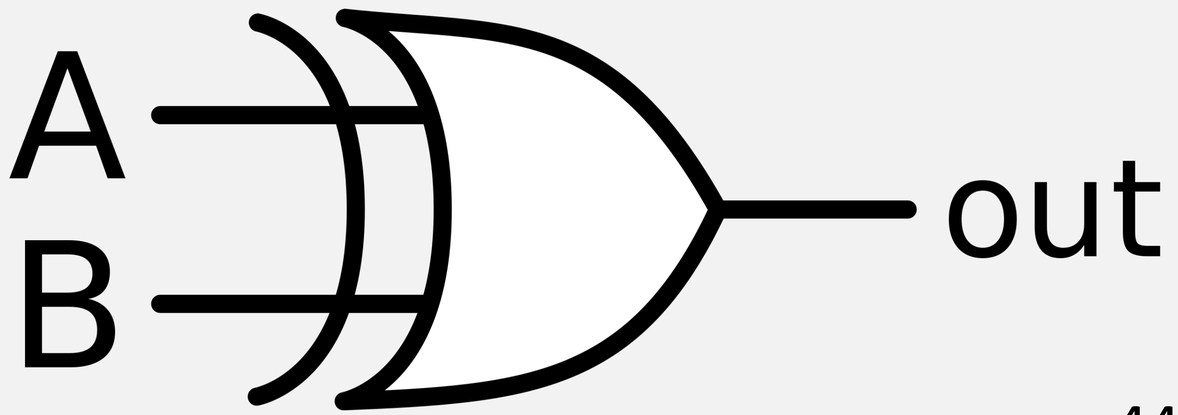
- XOR – (Exclusive Or) Output is true iff one input is true

  Alternatively true if odd number of inputs are true for >2 input XOR

- Notated as $A \oplus B$ *or* $A \cdot \bar{B} + \bar{A} \cdot B$

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	0



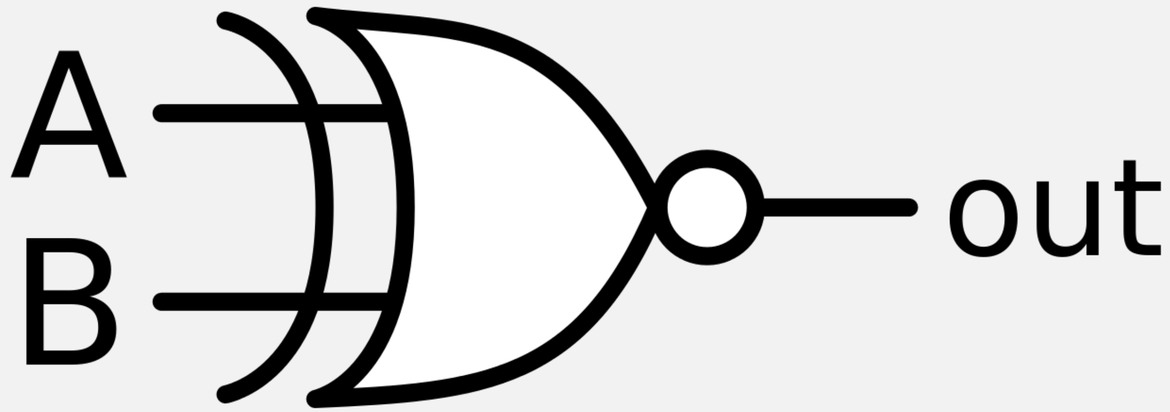


LOGIC OPERATIONS

- XNOR – (Exclusive Not OR) opposite of XOR where output is true if both inputs are false, or both inputs are true

- Notated as $\overline{A \oplus B}$ *or* $A \cdot B + \bar{A} \cdot \bar{B}$

A	B	Out
0	0	1
0	1	0
1	0	0
1	1	1

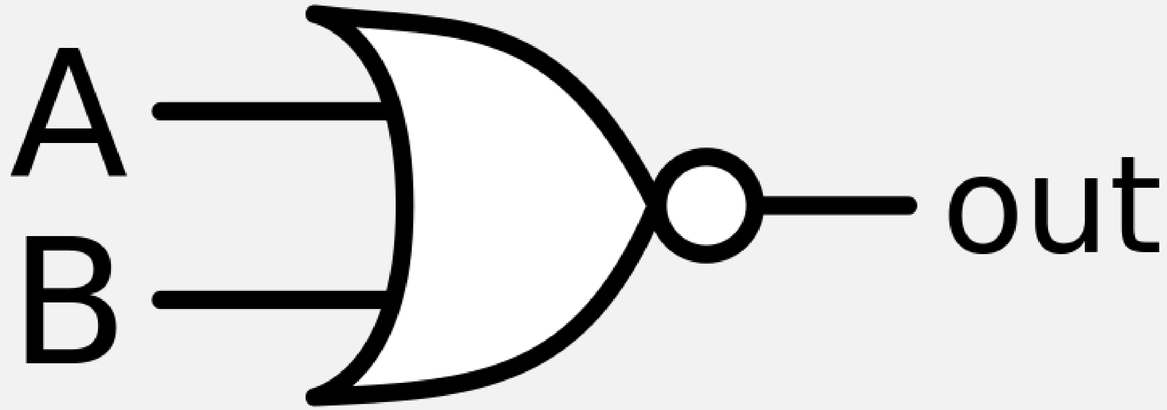


LOGIC OPERATIONS

- NOR – (Not OR) opposite of OR where output is true if neither input is true
- A universal gate – all other gates can be created from a NOR gate

Notated as $\overline{A+B}$

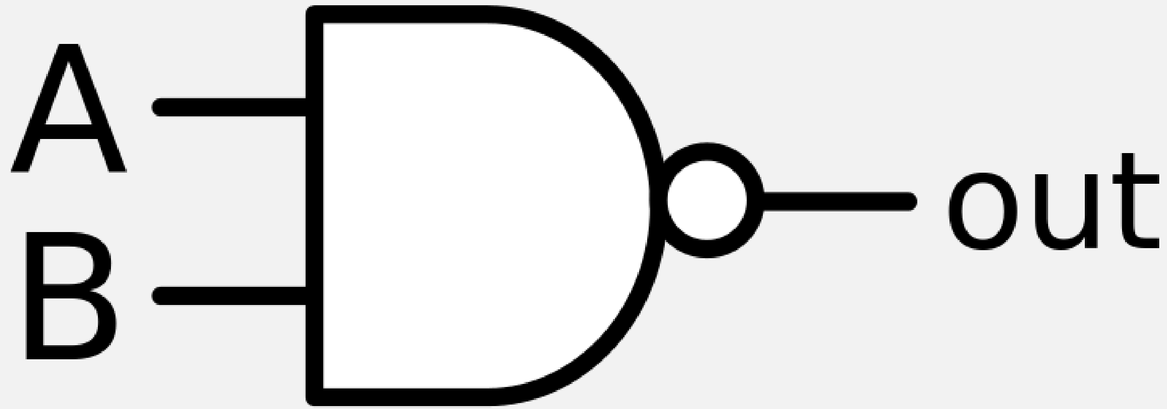
A	B	Out
0	0	1
0	1	0
1	0	0
1	1	0



LOGIC OPERATIONS

- NAND – (Not AND) opposite of AND where output is true if any input is false
- Also a universal gate –will show later
  - Notated as $\overline{AB}$ or $\overline{A.B}$

A	B	Out
0	0	1
0	1	1
1	0	1
1	1	0



REFERENCES

- [1] *What you never wanted to know about floating point but will be forced to find out*. [Online]. Available: https://www.volkerschatz.com/science/float.html. [Accessed: 26-Aug-2022].